

Exercises - low code version

1 - Model a Payment Process

Goal

The goal of this lab is to think of which activities and events belong to the payment process and model the process with Camunda Modeler.

Detailed Steps

1. Ensure you have a valid [Camunda SaaS account](#) set up for process execution, or sign up for a trial account if you still need one. If your trial is expired, ask your trainer to extend it for the duration of the training.
2. Discuss the requirements for payment in the order process.
 1. The customer can have an existing credit at the company.
 2. The amount should be deducted from this credit.
 3. If the customer has to pay more than his credit, then the credit card of the customer should be charged.
3. Model the payment process as a group using Camunda Web Modeler.
4. Discuss the process model(s) with the trainer afterwards and make sure every member of your group has access to the diagram.

2 - Create or Verify Access to a Cluster

Goal

Ensure you have access to a Camunda Platform 8 cluster and understand how authorizations might affect the next exercises.

Where to start?

- **You already have a cluster in the Camunda latest stable version?**
→ Skip creation and start from **Step 3**.
- **You are not sure what a cluster is, or don't have one yet?**
→ Start from **Step 1** and follow all steps in order.

Note: If you have a cluster on an older version, ask your trainer for guidance.

Detailed Steps

1. **Check for an existing cluster**
 - If you already have a cluster (for example, created when you signed up or provided by your organization), you can reuse it for this training.
 - If you do not have a cluster yet, create one in SaaS:
 - Go to the **Clusters** tab.
 - Click **Create New Cluster**.
 - Name your cluster, select the **stable** channel, the **latest** generation, and the region closest to you.
 - Click **Create**.
2. **Wait for the cluster to be ready**
 - Your cluster may take a few minutes to appear and become available.
 - Check the status on the **Clusters** page or by clicking on the cluster and viewing the **Overview**.
 - Continue once the status shows as *Healthy*.
3. **Be aware of cluster authorizations**
 - For **new SaaS clusters**, authorizations are **disabled by default**, so you can continue with the next exercises without changes.
 - For **existing or migrated clusters**, authorizations may be **enabled**:
 - If possible, **disable authorizations** in your cluster settings for the duration of the training.
 - If authorizations remain enabled, be aware that **next exercises may require additional permissions** in Operate, Tasklist, and other components. For background, see [Orchestration Cluster authorization](#).

- If you know or suspect that authorizations are enabled on your cluster, **raise this with your trainer** so they can support you during the following labs.

4. **Verify Operate access and running instances**

- Click on your cluster and open the **Overview** tab.
- Launch **Operate**.
- Check the number of running process instances; it should be 0 at this point for new clusters.

5. **Create an API client for the Orchestration Cluster API**

You'll need API credentials to connect the worker applications you'll create in further exercises to your Camunda cluster.

- Open Camunda Console and go to **Console → Clusters → your cluster → API** tab.
- Click **Create new client**.
- In the dialog:
 - **Name:** enter your client name e.g. `training-credentials`.
 - **Scopes:** select only **Orchestration Cluster API**.
- Confirm to create the client.
- When prompted, download the credentials file and store it securely. The download format depends on the programming language you'll use. If it's Java Spring then you can choose either **Spring boot** or **Spring boot yaml**, otherwise choose **Env Vars**. You'll use its values when calling the Orchestration Cluster REST API (v2).

3 - Deploy and Run a Payment Process

Goal

The goal of this lab is to deploy the previously created model to the cluster. Start a process instance and inspect the result in Operate.

Detailed Steps

Prepare your Diagram

1. In Web Modeler, make sure to switch to "Implement" Mode (top left). Give your process an ID (e.g. *PaymentProcess*) and a name (e.g. *Payment Process*) by clicking on the whitespace of the diagram and adding the values in the **General** section of the properties panel. Make sure that the checkbox **Executable** is checked.
2. The engine needs to be able to choose a direction after the XOR gateway(s). For each outgoing sequence flow after a splitting XOR gateway define a condition expression, e.g.:

None

```
orderTotal <= customerCredit
```

or

None

```
not(orderTotal <= customerCredit)
```

Deploy and run your Diagram from Web Modeler

1. To deploy the process model to your cluster, make sure you are still in *Implement* mode, then click **Deploy & Run** button in the top-right corner.
2. Select your Cluster and enter the required variables. An example input could look like this:

JSON

```
{ "orderTotal": 45.99, "customerCredit": 30 }
```

3. Click **Deploy & Run** again.

Inspect the Result

1. Go to Operate and have a look at the process instances.
2. Go to the tab **Processes**.
3. Add finished instances to your filter. You should now see one completed process instance.
4. Inspect the finished process instance by clicking on the process instance id. You can see the path that was taken and the process variables that were supplied as the payload.

4 - Model Service Tasks and Run them with REST

Prerequisite

Swagger-ui enabled on your Camunda 8.8 cluster (In SAAS it's enabled by default).

Goal

The goal of this lab is to

- Model Service tasks
- Start a process instance using REST
- Monitor the progress in Operate
- Activate and complete the jobs with REST.

Detailed Steps

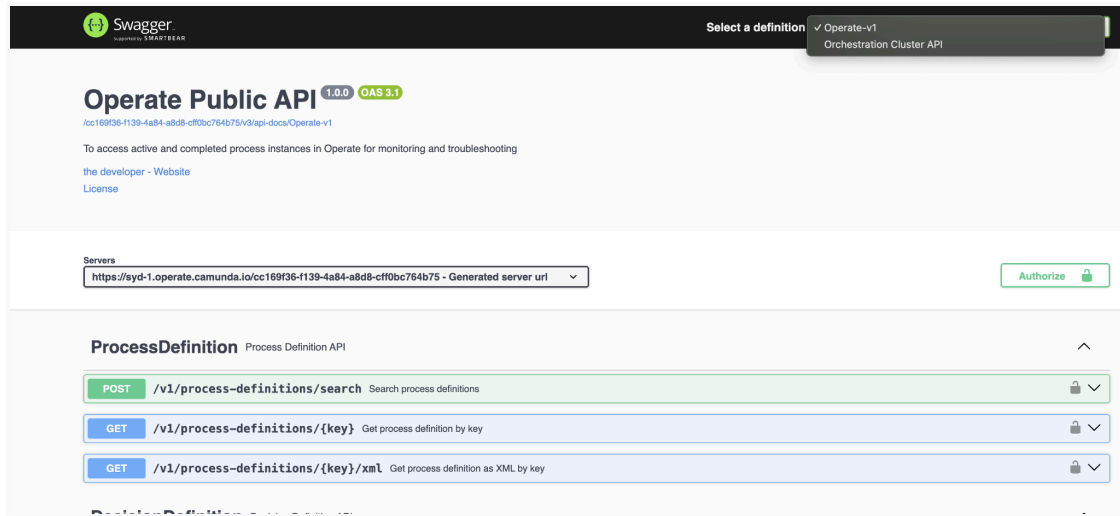
Process Modeling

1. Open the payment process in Modeler.
2. Change the type of each task to **Service Task** and define a **Task definition type** in the properties panel. The type should be a string (e.g. `credit-deduction`, `credit-card-charging`).
3. Deploy the process model to your cluster.

Swagger UI

We will use Swagger UI, a rest api client included by default in a Camunda cluster. The easiest way to access Swagger ui is the following:

- Login to Operate in order to authenticate.
- Change the last part of the url operate to `swagger-ui/index.html`
- Complete url:
`https://{REGION_ID}.operate.camunda.io/{CLUSTER_ID}/swagger-ui/index.html`



- click on the top right dropdown on **Orchestration Cluster API**: all the apis will be displayed.

Start process

1. Scroll down to the **Process instance** group
2. Click the POST `/v2/process-instances` end point
3. Click Try out
4. Configure the json as

```
JSON
{
  "processDefinitionId": "PaymentProcess",
  "processDefinitionVersion": -1,
  "variables": { "orderTotal": 45.99, "customerCredit": 30 }
}
```

By setting `processDefinitionVersion` to -1, we specify that the latest version of the process will start.

5. Click execute
6. Check the server response, you should get a 200 status, verify in Operate if the process instance has started and a token is waiting in the **Deduct credit** task.
This means that there's a new job to do. In this exercise we will complete the job with the REST apis. We need a first request to activate the job and a second one to complete it.

Activate job

1. Look for the Job group
2. Click the POST /v2/jobs/activation end point
3. Click Try out
4. Configure the json as

```
JSON
{
  "type": "credit-deduction",
  "worker": "myWorker",
  "timeout": 200000,
  "maxJobsToActivate": 1,
  "fetchVariable": [
    "orderTotal", "customerCredit"
  ],
  "requestTimeout": 5000
}
```

The response will contain the `jobKey` that is needed for completing the job. Copy it to the keyboard.

Complete job

1. Switch to the POST /v2/jobs/{jobKey}/completion end point
2. Click Try out
3. set the copied jobKey in the related field
4. The body can be empty for our use case, but let's just create a variable that will be saved in the process as process variable:

```
JSON
{
  "variables": {"testVar": "variable created when the job is completed"}
}
```

6. Once you complete the job, verify the updated status in Operate: the token should now be placed on the Charge credit card task.

Repeat for the other task

8. Now repeat the Activate/Complete requests for the `Charge credit card` service task, then the process instance should be finished.
9. Check the history of the process instance. How long did the instance take to complete?

5 - Create FEEL expressions for your tasks

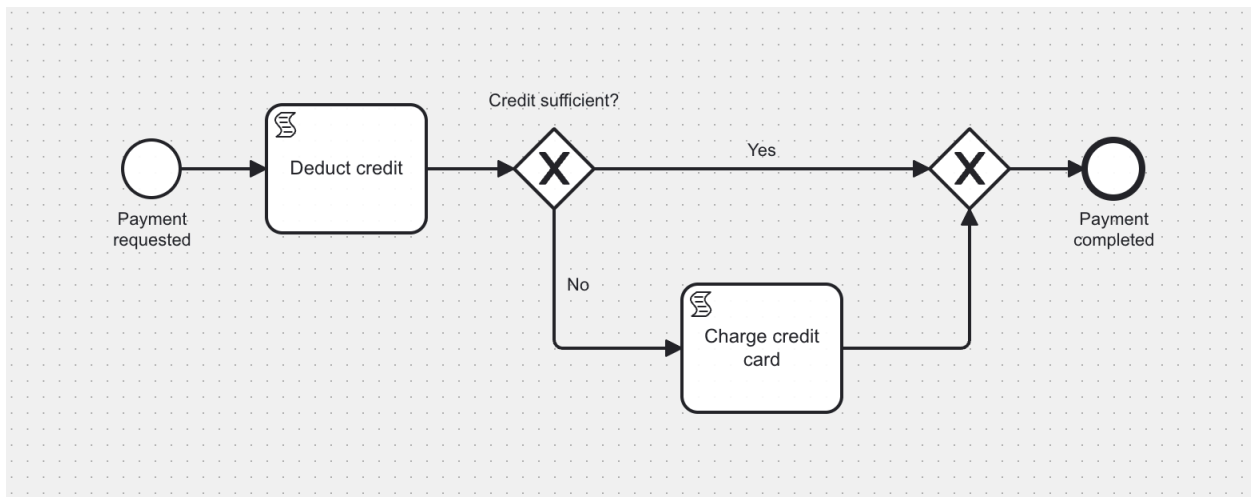
Goal

Implement FEEL expressions.

Detailed Steps

Script tasks can be used to execute **FEEL expressions**. In this first iteration you will just use them to create a new variable that logs the execution of the process.

1. Change both service tasks to script tasks



2. Open the first task and in the **Script** tab set the **Result variable** to a name (for example, **log**).
3. In the **FEEL expression** field, enter the following:

JavaScript

```
"credit-deduction task executed " + string(now())
```

4. Repeat the same configuration for the other task. In the **FEEL expression** field, use:

JavaScript

```
"credit-card-charging task executed " + string(now())
```

5. In **Operate**, you should now see a process variable called `log`.

Questions for you:

- Does the `log` variable have the expected value?
- What is its value after both tasks have been executed?
- Why, in this case, do you only see the value from the second task?

Advanced - Explanation and Improvement: Avoid Overwriting `log`

In the current implementation, both script tasks write to the same result variable `log`. Because the second task runs after the first, its FEEL expression **overwrites** the previous value of `log`. This is why, in Operate, you only see the value produced by the second task.

To avoid this overwriting and instead **accumulate a history of executions**, you can make `log` a **list of entries** and append to it in each script task.

For example, in the second task, you can use:

```
None
append(
  log,
  {
    "index": count(log),
    "timestamp": now(),
    "message": "credit-card-charging"
  }
)
```

- `log` is assumed to be a list (initialized earlier, or created as a list in the first task).
- `append(log, ...)` returns a new list with the new entry added.
- `count(log)` gives you the current number of entries, which you can use as an `index`.
- `timestamp` and `message` store when and what was executed.

But you need to change also the first task feel expression because log is now a list. One way to do it, is just wrapping the previous expression inside two brackets []:

JavaScript

```
[ "credit-deduction task executed " + string(now()) ]
```

6 - Add Data Handling to Your Job Worker

Goal

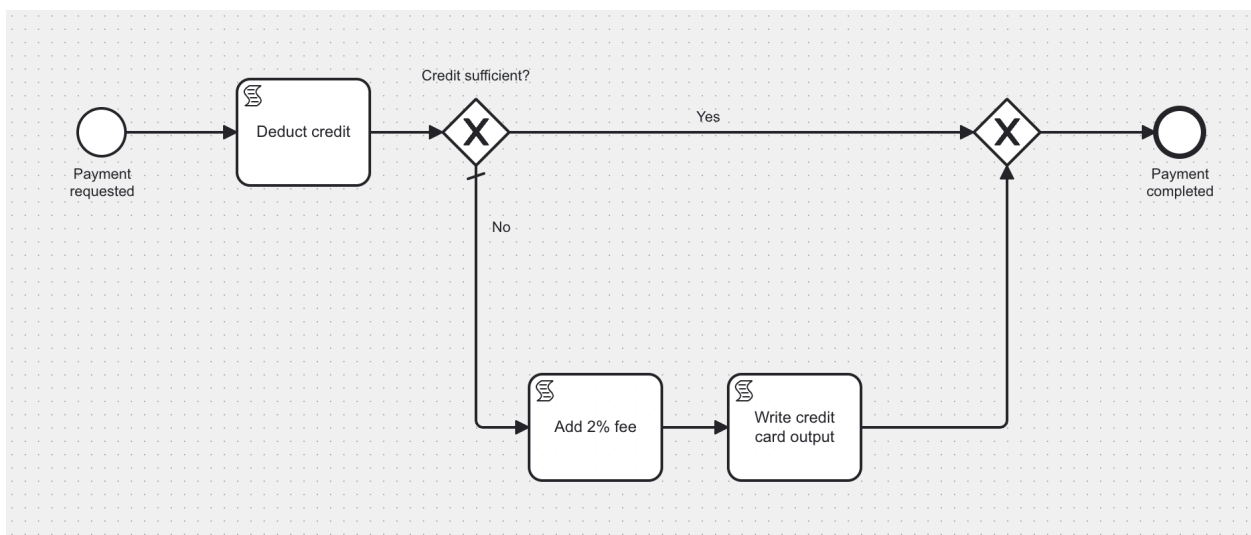
Improve your implementation of the payment process by handling process data using **FEEL expressions** only.

All implementation is done in **script tasks**, without external job workers.

Detailed Steps

1. Prepare the Diagram

1. Open the payment process in Web Modeler.
2. Add a script task to apply the credit card fee



2. Configure the “Deduct credit” Script Task

In this step, you calculate how much of the **order total** is still open after applying the customer’s credit.

1. Select the **Deduct credit** script task.
2. In the **Script** tab:
 - Set the Implementation to **FEEL expression**
 - Change the **Result variable** from **logs** to:
openAmount
3. In the **FEEL expression** field, enter the following expression:

None

```
orderTotal - number(substring(customerId, -2))
```

This expression assumes:

- **orderTotal** is a number representing the total order amount.
 - **customerId** is a string whose **last two characters** represent the customer’s credit (for example, **cust30** → credit **30**).
 - **substring(customerId, -2)** returns the last two characters of **customerId**, and **number(...)** converts them to a numeric value.
4. Adjust the **XOR gateway** conditions after *Deduct credit*:

Yes (credit sufficient) path:

None

```
openAmount <= 0
```

No (credit not sufficient) path:

None

```
openAmount > 0
```

3. Configure the “Add 2% fee” Script Task

If the credit is not sufficient, the process continues through the “No” path and adds a **2% fee** to the remaining amount.

1. Select the **Add 2% fee** script task.
2. In the **Script** tab:
 - Set **Script type** to **FEEL**.
 - Set **Result variable** to:
`openAmount`
3. In the **FEEL expression** field, enter:

None

```
openAmount * 1.02
```

This multiplies the remaining amount by **1.02** and overwrites `openAmount` with the fee included.

4. Configure the “Write credit card output” Script Task

In this step, you create a human-readable output message about the credit card charge.

1. Select the **Write credit card output** script task.
2. In the **Script** tab:
 - Update the **Result variable** from `logs` to:
`creditCardOutput`
3. In the **FEEL expression** field, enter:

TypeScript

```
string(openAmount)
+ " was charged from "
+ cardNumber
+ "; cvc: "
+ cvc
+ "; Expiry date: "
```

+ expiryDate

4. This expression assumes the following process variables are available:
 - **openAmount**: numeric value to be charged (possibly already including the fee).
 - **cardNumber**: credit card number as a string.
 - **cvc**: security code as a string.
 - **expiryDate**: expiry date as a string.
5. The result is a single string stored in **creditCardOutput**, for example:
6.
20 was charged from 1234 5678; cvc: 123; Expiry date: 09/28

5. Test the Process

1. **Deploy** the updated process to your cluster.
2. **Start a new process instance** from Web Modeler or via REST, with variables similar to:

JSON

```
{
  "orderTotal": 45.99,
  "customerId": "cust30",
  "cardNumber": "1234 5678",
  "cvc": "123",
  "expiryDate": "09/28"
}
```

3. Open **Operate** and inspect the process instance:
 - Check the updated value of **openAmount** after **Add 2% fee**.
 - Verify that **creditCardOutput** contains the expected text after **Write credit card output**.
4. Try changing the input variables (for example, **customerId** or **orderTotal**) and observe:
 - When does the process follow the **Yes** path (no credit card charge)?
 - When does it follow the **No** path and execute the fee and credit card output steps?

7 - Message Events

Goal

You will create a new **Order process** that triggers the existing **Payment process** by sending a message using the **Send Message connector**.

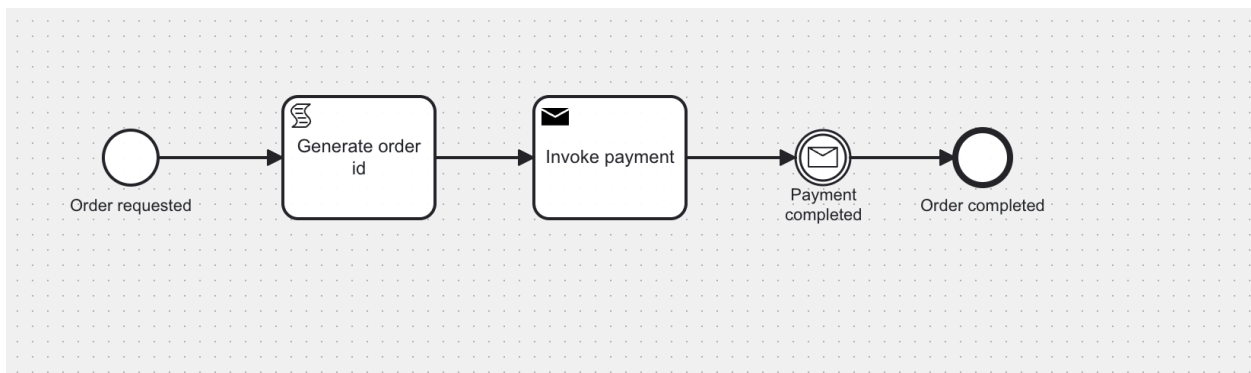
Detailed Steps

As part of this exercise you will:

- Model and configure the **Order process** so that it:
 - generates an `orderId`,
 - invokes the Payment process via a message,
 - waits for the payment to complete.
- Adapt the **Payment process** so it starts and completes by receiving and sending messages

1. Model the Order Process

1. Open Modeler and access the folder where your **Payment process** is located.
2. Create a new **Process application** called **Order Process**.
It will be generated in a new folder with the BPMN process.
3. Model an order process with the following elements (use these exact names in the diagram):
 - **Start Event:** *Order requested*
 - **Script Task:** *Generate order id*
 - **Send Message Connector:** *Invoke payment*
 - **Intermediate Message Catch Event:** *Payment completed*
 - **End Event:** *Order completed*



4. Fill in the technical attributes:
 - **Process Name:** Order Process
 - **Process ID:** OrderProcess
5. Configure the *Generate order id* script task:
 - **Result variable:** orderId
 - **FEEL expression:** uuid()

2. Adapt the Payment Process to Start and Complete by Message

The current **Payment process** starts with a **None Start Event** and ends with a normal **End Event**. In this step you will change it to **start** when a message is received and **complete** by sending a message back to the Order process.

1. In the same application, open your existing **Payment process**.
2. Change the **None Start Event** to a **Message Start Event**.
 - In the **Message** section of the properties panel:
 - Add a new **Global message reference**.
 - Set **Message name** to: paymentRequestMessage
3. Change the **End Event** to a **Send Message Connector (Message End Event)**.
 - Apply the **Send Message Connector (Message End Event)** template.
 - Configure the **Properties** as follows:
 - **Message name:** paymentCompletedMessage
 - **Correlation key:** orderId
 - The **payload** can be left empty
4. Save the Payment process.

When the Payment process finishes, it will now send a **paymentCompletedMessage** correlated by **orderId**, allowing the *Payment completed* message catch event in the **Order Process** to continue.

3. Configure the “Invoke payment” Send Task (Send Message Connector)

You will now configure the *Invoke payment* send task using the **Send Message connector**. All data will be passed in the **Payload**; there is no need for additional input mappings.

1. In the **Order Process**, select the **Send Task** named *Invoke payment*.
2. In the properties panel, set the implementation to **Send Message connector** (Send Message Connector (Send Task)).
3. Configure the connector **Properties** exactly as follows:
 - **Message name** to: paymentRequestMessage
 - **Payload:**



JSON

```
{  
  
  "orderTotal": orderTotal,  
  
  "customerId": customerId,  
  
  "cardNumber": cardNumber,  
  
  "cvc": cvc,  
  
  "expiryDate": expiryDate,  
  
  "orderId": orderId  
  
}
```

4. Configure the “Payment completed” Message Catch Event

1. In the **Order Process**, select the **Intermediate Message Catch Event** named *Payment completed*.
2. In the **Message** section of the properties panel:
 - Add a **Global message reference** if not already present.
 - Set **Name** to: `paymentCompletedMessage`
 - Set **Subscription correlation key expression** to: `orderId`

This will allow the Order process to continue when the Payment process later sends a `paymentCompletedMessage` correlated by `orderId` (configuring that outgoing message in the Payment process can be part of a follow-up or advanced step).

5. Deployment and Testing

1. **Deploy the Order Process application.**
Because the Payment process is part of the same application, it will be deployed automatically together with it.
2. Deploy and Start a new process instance of **OrderProcess** with a payload like:



JSON

```
{  
  
  "orderTotal": 45.99,  
  
  "customerId": "cust30",  
  
  "cardNumber": "1234 5678",  
  
  "cvc": "123",  
  
  "expiryDate": "09/28"  
}
```

3.

Open **Operate** and inspect the running instances:

- The **Order Process** instance should:
 - start at *Order requested*,
 - generate an `orderId` in *Generate order id*,
 - send the message at *Invoke payment*.
- A **Payment process** instance should be started by the **Message Start Event** using the `paymentRequestMessage` and the connector payload (including `orderId`).

Once the Payment process is configured to send back a `paymentCompletedMessage` correlated by `orderId`, the Order process will continue at *Payment completed* and finish at *Order completed*.

Questions for you

- Which element in the **Order Process** actually triggers the **Payment process**?
- Which variables are included in the **Send Message connector** payload, and where does each one originate?
- Why do you not need to provide `orderId` when starting the Order process?
- How could you extend this pattern if you wanted to send additional information (for example, shipping address or discount) to the Payment process?

8 - Catch Incident and Handle Failure with FEEL

Goal

In this exercise you will **force an incident** in the payment process and then resolve it using **Operate**.

Detailed Steps

You will:

- Add a **FEEL assertion** in the *Write credit card output* script task to validate the credit card expiry date.
- Trigger an incident when the expiry date has an invalid format.
- Fix the data in Operate and **retry** the incident.

1. Add a FEEL Assertion to Validate the Expiry Date

1. Open your **Payment process** in Web Modeler.
2. Select the **script task** named **Write credit card output**.
3. Create a new **Input variable** for this task:
 - **Name:** expiryDateCheck
 - **FEEL expression:**

JavaScript

```
assert(
  expiryDate,
  string length(expiryDate) = 5,
  "The expiry date must have exactly 5 characters (e.g., MM/YY)"
)
```

- `expiryDate` is the variable we want to return if the assertion is verified.
- `string length(expiryDate) = 5` checks that the value has exactly five characters.
- If the condition is **false**, the `assert(...)` function throws an error with the given message, causing an **incident** in the *Write credit card output* task.

Your original FEEL script to build `creditCardOutput` can stay as it is; the new `expiryDateCheck` input is only there to validate the data before or during the task execution.

Save and redeploy the process application.

2. Trigger an Incident with Invalid Expiry Date

1. Start a new process instance (either directly on the Payment process or via the Order process, depending on your previous exercises) with variables similar to:

```
JSON
{
  "orderTotal": 45.99,
  "customerId": "cust30",
  "cardNumber": "1234 5678",
  "cvc": "123",
  "expiryDate": "09/28x"
}
```

2. Note that expiryDate is **invalid** here ("09/28x" has 6 characters instead of 5).
3. Open **Operate** and locate the process instance.
4. Drill down to the active element.
 - You should see an **incident** on the **Write credit card output** script task.
 - In the incident details, you should see the error message from the FEEL assertion:
The expiry date must have exactly 5 characters (e.g., MM/YY)

3. Repair the Incident and Retry

1. In **Operate**, open the **Variables** view for the incident process instance.
2. Edit the expiryDate variable and set it to a valid value, for example: 09/28
3. Click **Retry** to increase the number of retries.
 - The process engine will re-execute the **Write credit card output** task.
 - This time, `string length(expiryDate) = 5` evaluates to **true**, so the `assert(...)` call does **not** fail.
 - The incident disappears and the token moves on.
4. Refresh the instance view in Operate:
 - The **Write credit card output** task should now be completed successfully.
 - The process instance should continue and eventually reach the end event.

Questions for you

- What happens to the process instance when the `assert(...)` condition fails?
- Which variable did you have to change in Operate to repair the incident?

- How could you extend this FEEL assertion to also check that the expiry date contains a / in the middle (e.g., "MM/YY")?

9 - Handle BPMN Errors with Low Code

Goal

In this exercise you will handle **BPMN errors** when the expiry date is not valid

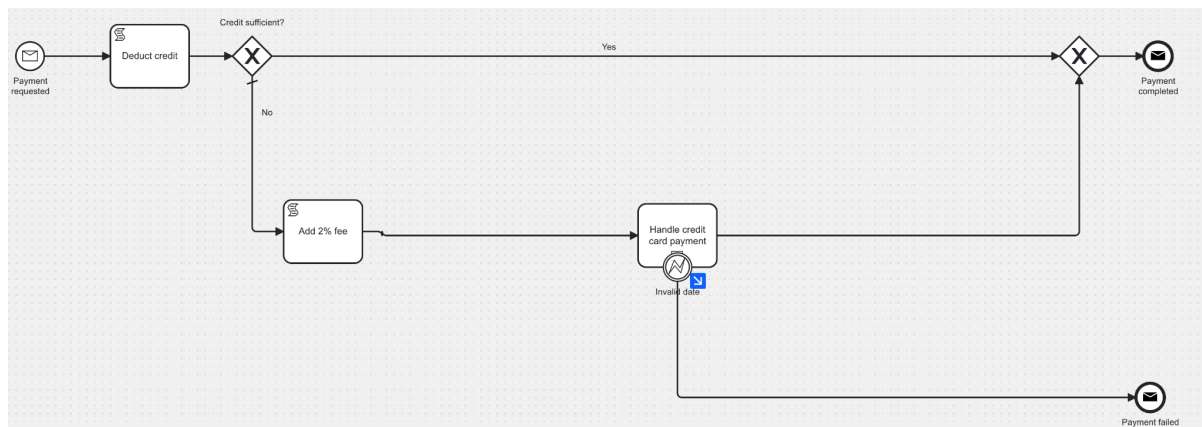
Detailed Steps

You will:

- Extend the **Payment process** to throw a **BPMN error** when the credit card expiry date is invalid.
- Catch this error with a **boundary error event** and send a failure message (paymentFailedMessage) back to the **Order process**.
- Use an **Event-based gateway** in the Order process to react differently to successful and failed payments.

1. Extend the Payment Process to Throw a BPMN Error

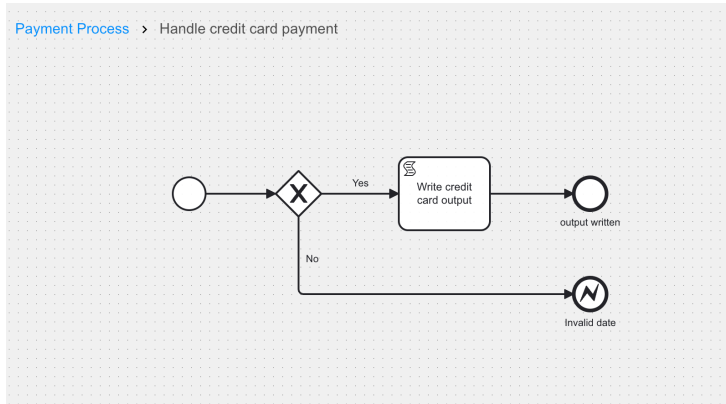
1. Open the **Payment process** in Web Modeler.
2. After the **Add 2% fee** script task, create a new **collapsed subprocess** and move the **Write credit card output** task into it. You can do this by clicking on the blue arrow and switching to the subprocess view.



In the main process you need to :

3. add a **Boundary Error Event** attached to the subprocess task, labeled *Invalid date*, connected to a new **Message End Event** *Payment failed* (use the send message connector as done previously).
4. Configure the **Boundary Error Event**, in the **Error** section:
 - Create or select a **Global error**, e.g. *Invalid Date*

- **Error code:** `invalidDate`



Entry into collapsed subprocess:

- **Start event** inside the subprocess.
 - **Exclusive gateway** (data-based) to check if the expiry date is valid.
 - **Yes** path → **Script task**: *Write credit card output* → **End event**: *output written*.
 - **No** path → **Error End Event**: *Invalid date* (throws a BPMN error).
 - Script task (unchanged)
- Configure the **exclusive gateway** inside the subprocess:
 - Select the gateway.
 - For the **Yes** outgoing sequence flow (the path to *Write credit card output*), set the **FEEL condition** to: `string length(expiryDate) = 5`
 - For the **No** outgoing sequence flow (the path to the *Invalid date* error end event), either:
 - Leave it as the **default flow**, or
 - Use the negated condition: `not(string length(expiryDate) = 5)`
 - Configure the **Error End Event** *Invalid date*, in the **Error** section:
 - Use the same **Global error** as above.
 - **Error code:** `invalidDate`

This ensures that when the inner *Invalid date* error end event is reached, the boundary event catches the BPMN error and triggers the alternative flow.

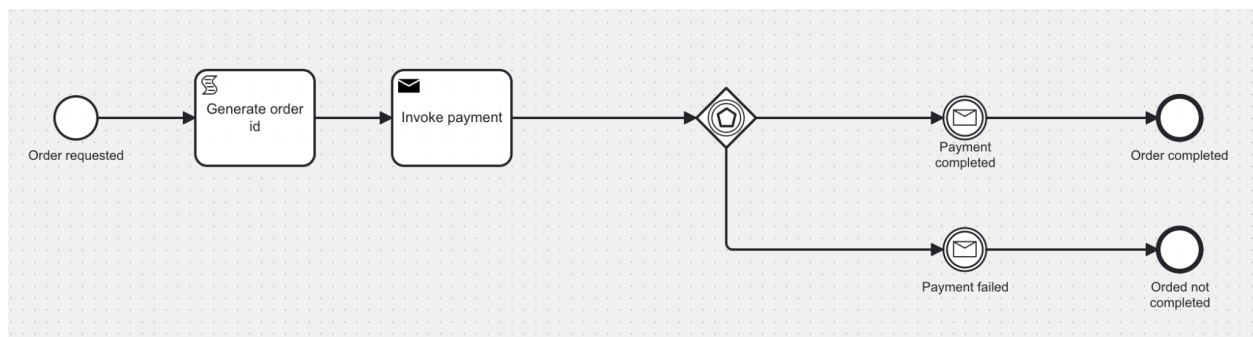
7. Configure the **Message End Event** *Payment failed* (Send Message Connector):

- Set the properties as follows:
 - Message name: `paymentFailedMessage`
 - Correlation key: `orderId`
 - Payload: you can leave it empty for this exercise.

2. Adjust the Order Process to React to Success or Failure

The Payment process can now also send a failure message, and our Order process must react accordingly.

8. Add an event-based gateway between the `Invoke payment` task and the `Payment Completed` message catch event.
9. Connect the gateway to another message catch event placed below the existing one, and name the message `Payment Failed`.
10. End that path with an end event named `Order not completed`.



11. Configure the `Payment failed` message by adding these parameters:

Python

```
Global message reference = paymentFailedMessage  
Subscription correlation key = orderId
```

12. Save the Order process and **deploy the Order Process application**.

The Payment process is deployed together with it as part of the same application.

3. Test the BPMN Error Flow

13. Start a new **OrderProcess** instance with an **invalid expiry date** to trigger the BPMN error:

JSON

```
{  
  "orderTotal": 45.99,  
  "customerId": "cust30",  
  "cardNumber": "1234 5678",  
  "cvc": "123",  
  "expiryDate": "09/28x"  
}
```

14. In **Operate**, inspect the **Order process** and **Payment process** instances:

- For the **Order process**, the instance should end up on the **Payment failed** path after the event-based gateway.
- For the **Payment process**, the instance should follow the **error path** (invalid date), be caught by the boundary error event, and finish by sending the `paymentFailedMessage`.

Questions for you

- Where in the **Payment process** is the BPMN error with code `invalidDate` thrown, and how is it caught?
- How does the **Event-based gateway** in the **Order process** decide whether to follow the *Payment completed* or *Payment failed* path?
- In which situations would you prefer using a **BPMN error** over creating an **incident** (as in the previous exercise)?

10 - Add Task Forms and Use the Camunda Tasklist

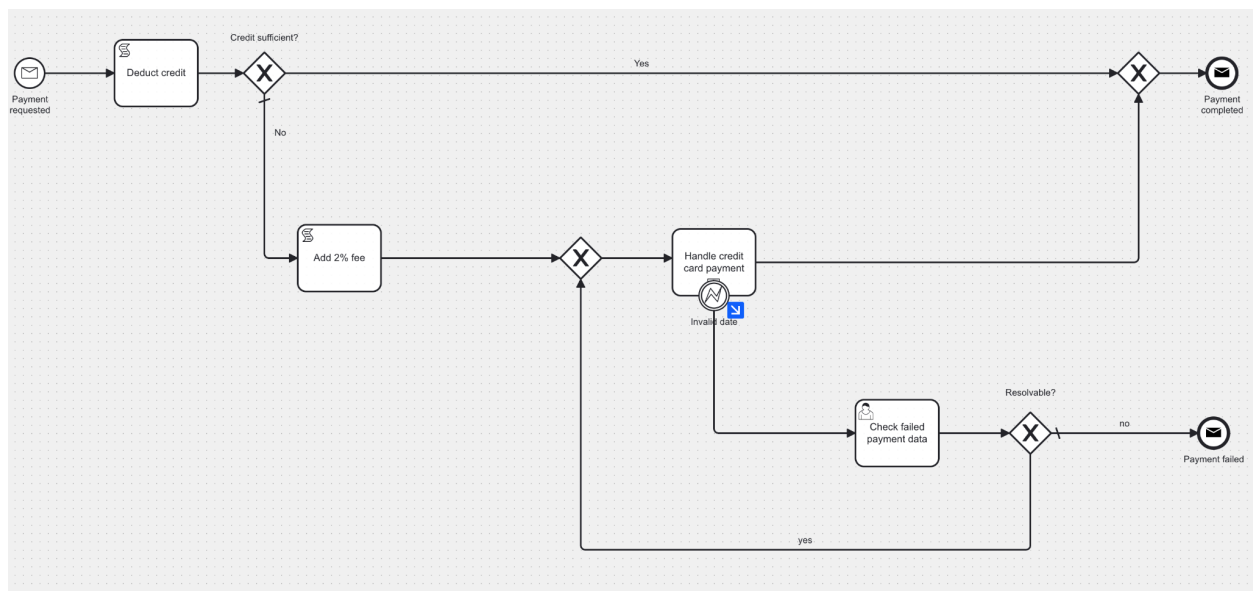
Goal

Sometimes the error can be resolved with manual interaction. In this lab you will add a user task with a simple form to your process and define what data should be displayed on it.

Detailed Steps

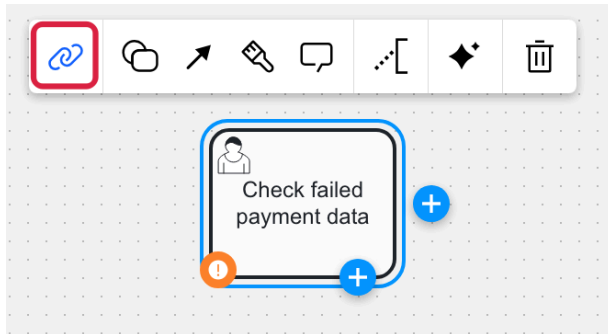
Process Modeling

1. In case the payment failed, a user should check the data and correct it, if possible. If the data could not be corrected to get the payment, the process continues with the error handling.
2. Open the payment process in Modeler. Add a task to the outgoing sequence flow of the error boundary event. Label the task like **Check failed payment data**. Change the task type to **User Task**.
3. Add an exclusive gateway after the new user task and before the **Fail Payment** task. Label the gateway with a question such as *Resolvable?*.
4. Set the “no” outgoing sequence flow as the default flow.
5. Connect that sequence flow with the *Payment failed* end event.
6. Add a condition and “yes” label to the other sequence flow which leads back to the *Charge credit card* task. Put simply the variable name (since it's boolean): `errorResolved`
7. Merge the flows with another XOR gateway.



Form Modeling

1. In the Web Modeler, select the "Check failed payment data" task and click on the link icon (first from the left).

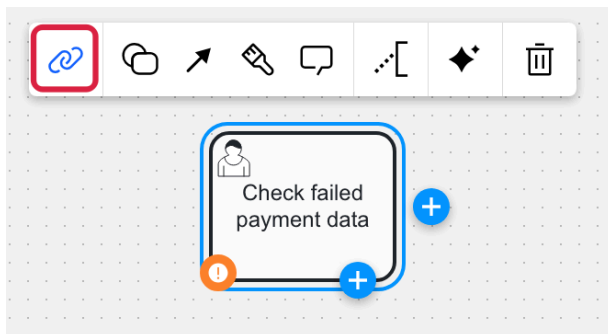


2. A popup window will appear. Click on *Create new form* (in blue on the bottom): The form editor will show up.
3. Drag a **Text** element from the palette onto the form canvas to provide a headline. The text could look like this:

None

Check the failed payment

4. You want to show in the form some process variables, like cardNumber, cvc and expiryDate. If you are unsure of the names you used, you can get the last process instance executed from *Operate*, and check in the Variables tab.
5. Add a **Text Field** for each String type variable. Fill in the **Field Label** with a human readable name. Also edit the **Key** section to match the respective process variable names.
6. Add a **Number** input for each Double type variable (orderTotal & openAmount).
7. Add a **Checkbox** for the user decision. Its **Key** should be **errorResolved** to match the conditions of the XOR gateway.
8. Go back to the Payment process diagram and Select the **UserTask** you are working on. Click again on the link icon.



9. The form you have just created will be available here and visible in a small preview.

Acceptance Test

1. Deploy and start a new instance of the order process. You can use the payload from exercise 8 with an invalid credit card.

JSON

```
{
  "orderTotal": 45.99,
  "customerId": "cust30",
  "cardNumber": "1234 5678",
  "cvc": "123",
  "expiryDate": "09/28x"
}
```

2. Open Tasklist (**camunda.io** -> **Clusters** -> **Your Cluster** -> **Launch Tasklist**).
3. Once all tasks are completed and the payment failed, you will see a user task appearing on the left side.
4. Select the task and inspect the data on the form.
5. In case you miss data or see any errors, you can modify the form in Modeler and redeploy it. If you refresh the page in your browser the Tasklist view will update to the latest version of the form.
6. Claim the task in the Tasklist. You can now edit the values.
7. To test a **successful retry**, correct the **expiryDate** (for example to **09/28**), set the **errorResolved** checkbox to true, and complete the task so the process retries the payment and follows the Payment completed path.
8. To test the **fail-payment** behaviour, decide the payment cannot be fixed, leave the **errorResolved** checkbox unchecked, and complete the task so the process follows the Payment failed path without retrying.

Questions for you

- How does the value of the errorResolved checkbox influence which path the process takes after the “Check failed payment data” user task?
- In which situations would you send the process back to retry the payment, and in which situations would you deliberately choose the “Payment failed” path?
- Which variables on the form are essential for a user to decide whether the payment can be resolved, and why?

11- Use a connector to get information about the product you want purchase.

Goal

Add a connector in order to get additional information about the product you want to purchase

Detailed Steps

Important: At the moment this exercise can be completed only with the Camunda **Web Modeler**. If you have created your diagram with the Desktop Modeler you should export it and import it in the Web Modeler.

Let's get some information about the product the user wish to purchase. We'll start the process with a new variable (**productId**), and we'll use it to query an external service and get some data about the product. For this exercise we use a popular fake api provider called [DummyJson](#)

Process Modeling

1. Open the **Order process** in the Web Modeler.
2. Add a task after the **Generate order ID** Script task, you can call it *Fetch product info*
3. Change the task type to *Rest Outbound Connector*

Token creation and storage

We need a token to access our product service which is security protected. Once we get the token we'll store it in our cluster as a **Secret**

4. DummyJson service provides a token authentication to manage security, you can check the documentation [here](#)
5. Use a rest api client of your choice to do the following call:

JSON

```
url: "https://dummyjson.com/auth/login"
method: "POST",
headers: { "Content-Type": "application/json" }
body: { "username": "emilys", "password": "emilyspass" }
```

Example with curl:

Shell

```
curl -X POST " https://dummyjson.com/auth/login"  
-H "Content-Type: application/json"  
-d '{"username":"emilys","password":"emilypass"}'
```

6. This call should return a token, you can copy it to your clipboard.
7. Open The Camunda Web Console and Navigate to your Cluster.
8. Create a new **Connector Secret** for your cluster, give it a name (i.e. DUMMYJSON_SECRET) and paste the value you copied previously. Check [here](#) if you need detailed information about secrets creation.

Connector configuration

9. Let's go back to the Web Modeler and select the *Get product information* task in order to configure the connector
10. Choose the authentication type to **Bearer token**
11. In the **Bearer token** section insert the secret with this syntax:
{{secrets.DUMMYJSON_SECRET}} where *secrets* is a keyword and *DUMMYJSON_SECRET* is the secret name you chose before
12. HTTP Endpoint method will be "GET"
13. As for the **URL**, click on the "fx" symbol to activate the dynamic value mode. Insert in the text box the following value:

Java

```
"https://dummyjson.com/auth/products/" + productId
```

where productId is a new variable we'll provide at startup.

14. In the **Result expression** field we can parse the response and store the product information, for example:

JSON

```
{  
  "productName":response.body.title,  
  "productDescription":response.body.description,  
  "productPrice":response.body.price  
}
```


Since the service is returning the product price, we don't need to pass the order total as an input variable any more, we'll need to pass the productId.

Our worker is still expecting a variable called orderTotal though.

15. Update the payload of the Invoke payment task to map the orderTotal to the new productPrice variable.

Payload *fx*

```
= {  
  "orderTotal": productPrice,  
  "customerId": customerId,  
  "cardNumber": cardNumber,  
  "cvc": cvc,  
  "expiryDate": expiryDate,  
  "orderId": orderId  
}
```

Acceptance Test

16. Deploy the updated process and start a new process instance. You can use the following payload: The productId must be a string (surrounded by double quotes)

JSON

```
{  
  "customerId": "cust10",  
  "cardNumber": "1234 5678",  
  "cvc": "123",  
  "expiryDate": "09/28",  
  "productId": "3"  
}
```

17. Open Operate and check the instance of your Order process. Did everything work as expected? Can you see the process variables related to the product you selected?

12 - Add a Decision Table About Discount

Goal

Add a business rule task to the order process and calculate a discount for the order based on the order amount.

Decision Modeling

1. Open the process application folder and create a new **DMN diagram**.
2. Click on the decision table and set the **Name** to **Order Discount** and the **ID** to **orderDiscount**.
3. Open the decision table by clicking on the table icon in the top left.
4. Double click on the input column header and name the input column **Product price**.
5. Set the **Expression** to **productPrice**. Select **number** as **Type** to match the type of your process variable.
6. Name the output column like **Discount percentage** and set the **Output Name** to **discount** Set the variable **Type** to **number**.
7. Add rules to discount orders depending on the order amount. Use **Unique** as the **Hit Policy**. This ensures that only one rule may match.

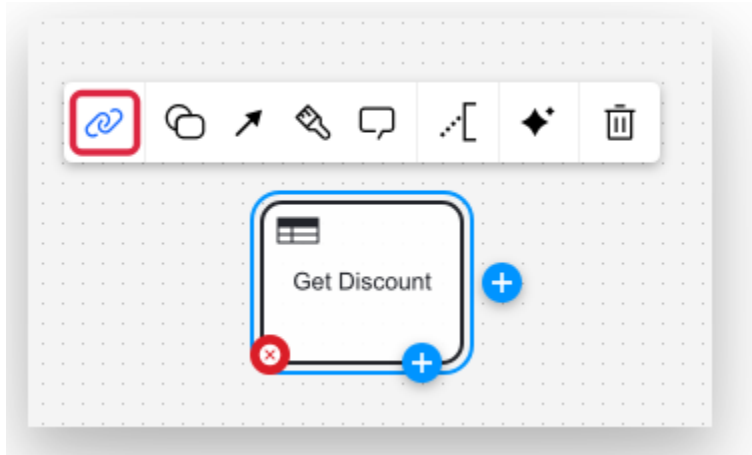
Here you are an example of the Decision Table

Order discount		Hit Policy:	Unique		
	When		Then	Annotations	
	Order amount	+	Discount percentage	+	
	number		number		
1	< 40.00		0		
2	[40.00 .. 60.00 [		2		
3	[60.00 .. 100.00 [		3		
4	>= 100.00		4		
+	-				

Process Modeling

Business Rule Task

1. Open the order process in Modeler.
2. Add a task to the order process to get the discount for the order amount before the invocation of the payment. Label the task **Determine discount**. Change the task type to **Business Rule Task**.



3. Select the task and click the link icon.
4. Choose the just created *Decision*.
5. Configure the **Result variable** for the Business rule task: In the **Implementation** panel under the property panel, enter **discount** in the Result variable field.

Apply the discount with a Script task

1. Add a task between the *Determine discount* and *Invoke payment* tasks to apply the discount to the order amount. Name the task **Subtract discount**.
2. Change the task type to **Script Task**. Add a feel expression that calculates the discount in the following way:

None

```
productPrice - (productPrice * discount / 100)
```

3. Name the **Result variable: discountedAmount**.
4. To have the customer pay only the discounted amount, update the payload in the *Invoke payment* task and map the orderTotal to the discountedAmount.

Acceptance Test

1. Deploy and start a new process instance. You can try different productId to test your DMN rules if needed.

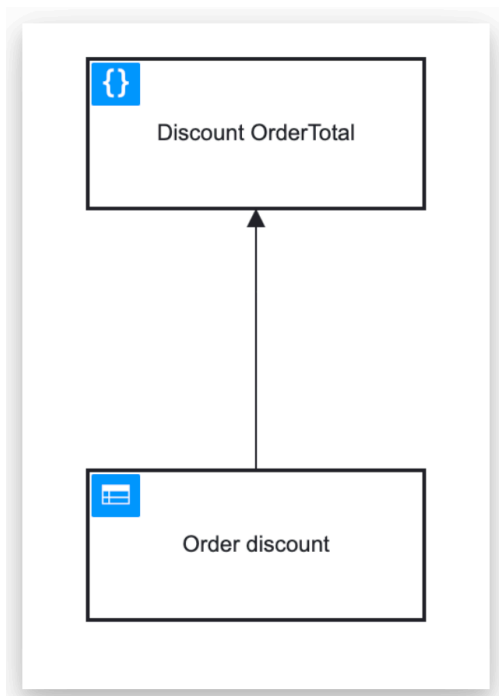
JSON

```
{
  "customerId": "cust10",
  "cardNumber": "1234 5678",
  "cvc": "123",
  "expiryDate": "09/28",
  "productId": "3"
}
```

2. Switch to your IDE and inspect the console log. You should see the output for all workers.
3. Open Operate and check the history of the order and the payment process. Was the discount applied correctly?
4. To further verify the evaluation of your decision table, go to the **Decisions** tab where you can see all (past) decision instances.

OPTIONAL: Another way to calculate the discount

In the previous steps, we calculated the discount using a script task. However, there's another option. Instead of using a *script task*, we can utilize the Decision diagram and add a **Literal expression** as shown in the following picture:



1. Insert the discount logic with a **FEEL** expression in the *Discount Order Total* Literal expression as follows:

None

```
round half down (productPrice - (productPrice * orderDiscount / 100),  
2)
```

2. Provide a suitable name and type for the Variable.

Additional exercises

A - Approve high value contracts

(multi-instantiation, listeners)

Goal

Design and run a process that coordinates the review of a high-value contract by several named approvers. The process should start with basic contract data (title and value) plus a list of people who must review it.

Each of those people should receive their own review task showing the contract information and be able to complete their part independently.

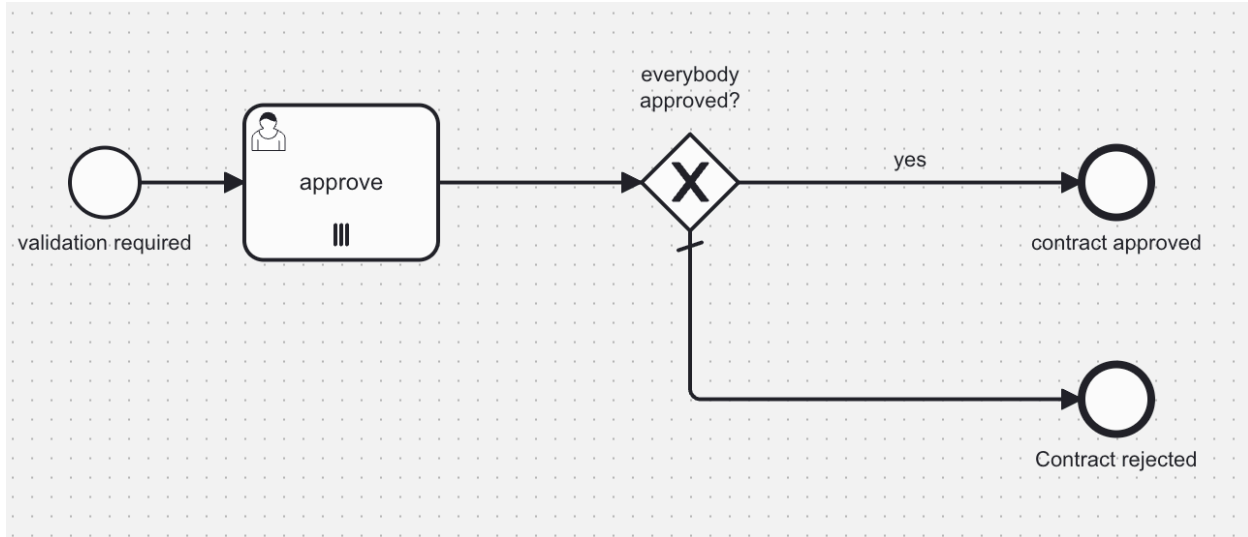
A sample JSON payload to start the process:

None

```
{  
  "contractTitle": "Enterprise Support Agreement 2026",  
  "contractValue": 250000,  
  "approvers": [  
    "alice@example.com",  
    "bob@example.com",  
    "carol@example.com"  
  ]  
}
```

Detailed steps

1. Model a process with:
 - Start event
 - User task **“Review contract”**
 - End event (or later, a gateway to collect/aggregate decisions if you want)
-



2. Configure the **“Review contract”** task so that the platform creates one review task for each person listed in the `approvers` variable.
3. Create a **form** for the “Review contract” task that:
 - Displays `contractTitle` (read-only text)
 - Displays `contractValue` (read-only number)
 - Contains a field to capture the decision, e.g. a boolean `approved` (checkbox or radio)
4. **Deploy and start** the process from Web Modeler using **Deploy & Run**, providing a payload like the sample JSON above.
5. Use Tasklist and/or the API to verify that:
6. Each approver receives a **“Review contract”** task.
 - The form shows the expected contract data.
 - Each approver can set `approved` and complete their task independently.
7. **(Optional – for trainees with a worker application)**
 - Add a user task listener on the **assignment** of “Review contract”.
 - Implement a worker that handles these listener jobs and writes a log entry such as:
`"[EMAIL] Assigned review of '<contractTitle>' to <assignee>"` whenever a review task is assigned.
 - Re-run the process and confirm that a log line appears for each assigned review task.

Questions for you

- If the platform creates all the review tasks for the different approvers **in parallel**, how will the reminder log lines appear compared to when it creates and assigns the review tasks **one after another (sequentially)**?
-